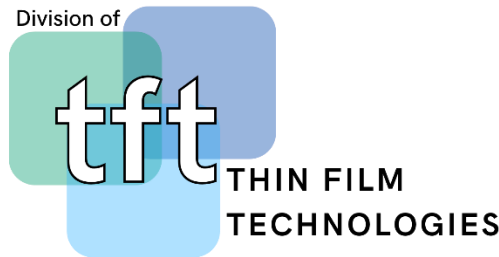




Politechnika
Wrocławska



android Android

Interfejs użytkownika (UI)

"Oszałamiający interfejs (UI) nic nie znaczy, jeśli koncepcja twojego produktu jest pozbawiona sensu. Piękny przemysłowy design może sprzyjać sprzedaży produktu w krótkim okresie czasu, ale nie zamaskuje okropnego serwisu"

(C. Rowland, M. Charlier, User Experience Design for the Internet of Things, O'Reilly Media, USA, 343 2015, s. 24.)

Przy IU pomocna może być strona <https://m3.material.io/>, która pomaga w tworzeniu ładniejszych i dopracowanych aplikacji.

Przydatna może być też aplikacja ze sklepu Google – Android Material Component. <https://play.google.com/store/apps/details?id=com.eunidev.materialdesign&hl=pl&gl=US>

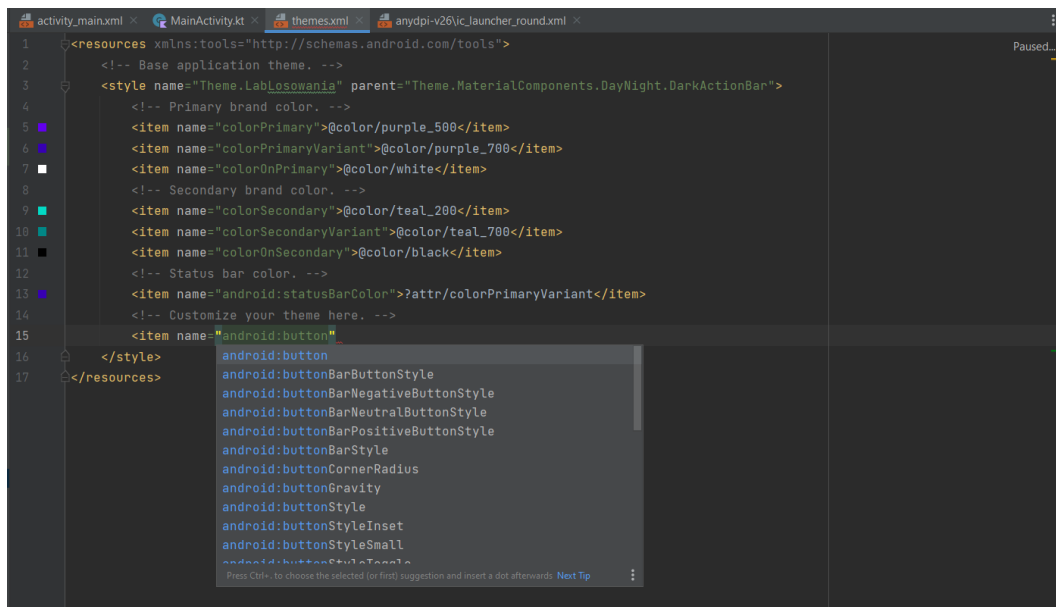
1. Zmiana kolorów tematu/motywu

Motyw/temat to zbiór atrybutów, który jest stosowany do całej aplikacji, aktywności lub hierarchii widoków. Po zastosowaniu motywu każdy widok w aplikacji lub aktywności stosuje wszystkie atrybuty motywu, które obsługuje. Motywy mogą również stosować style do elementów niebędących widokami, takich jak pasek stanu i tło okna. Innymi słowy, to **zbiór określonych, nazwanych atrybutów (zasobów), do których można później się odwoływać za pomocą stylów, layoutów itd. w całej aplikacji.**ⁱ

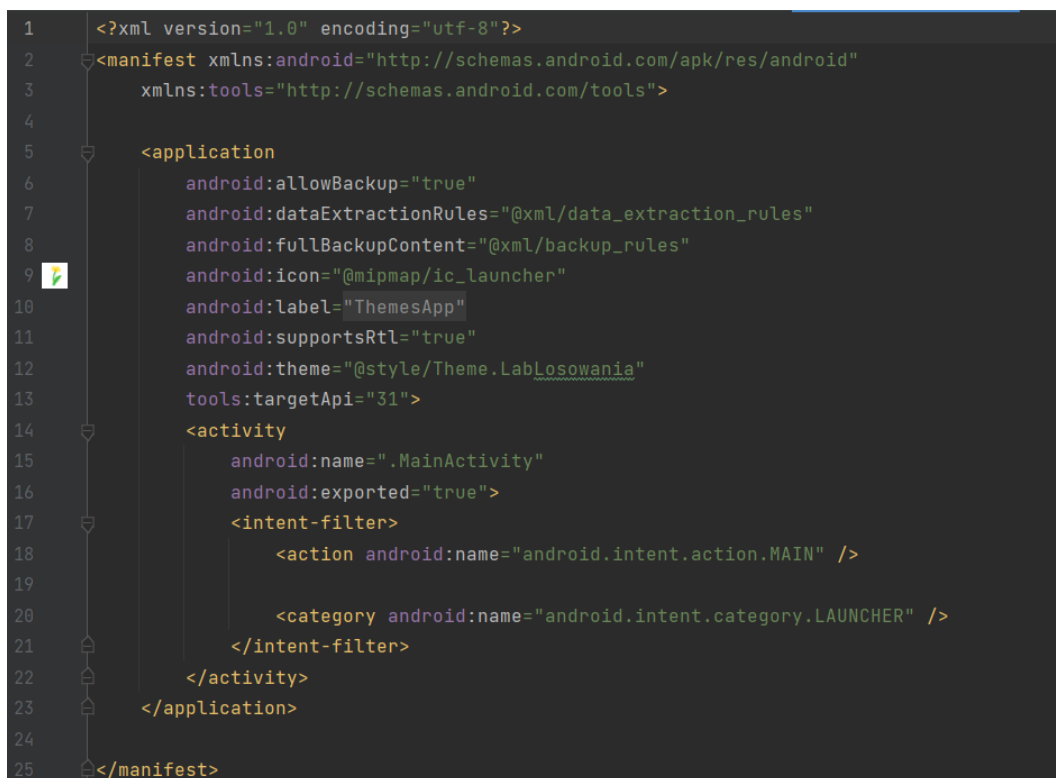
Styl z kolei jest zbiorem atrybutów, które określają wygląd pojedynczego widoku. Styl może określać takie atrybuty jak kolor czcionki, rozmiar czcionki, kolor tła i wiele innych.

Tematy/motywy

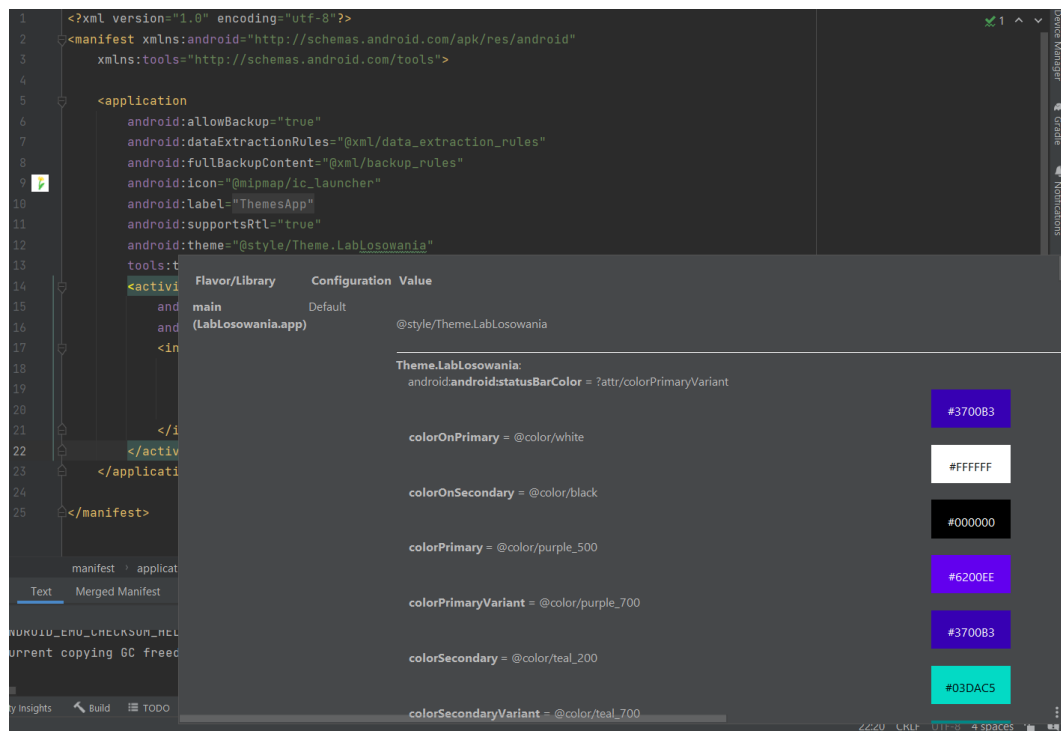
Kolory i inne atrybuty tematów można zmieniać w pliku *res* → *values* → *themes* → *themes.xml*. Domyślnie utworzone są dwa pliki *themes*, gdzie każdy z nich przyporządkowany jest do jednego trybu wyświetlania – dziennego lub nocnegoⁱⁱ. W tym pliku można zmieniać atrybuty każdego komponentu w aplikacji, na przykład zdefiniować kolory podstawowe oraz dodatkowe w aplikacji, ustawić domyślną czcionkę dla całego tekstu lub też zmienić domyślny kolor tła przycisków (jak poniżej).



Temat domyślny jest zawarty w pliku Manifest:



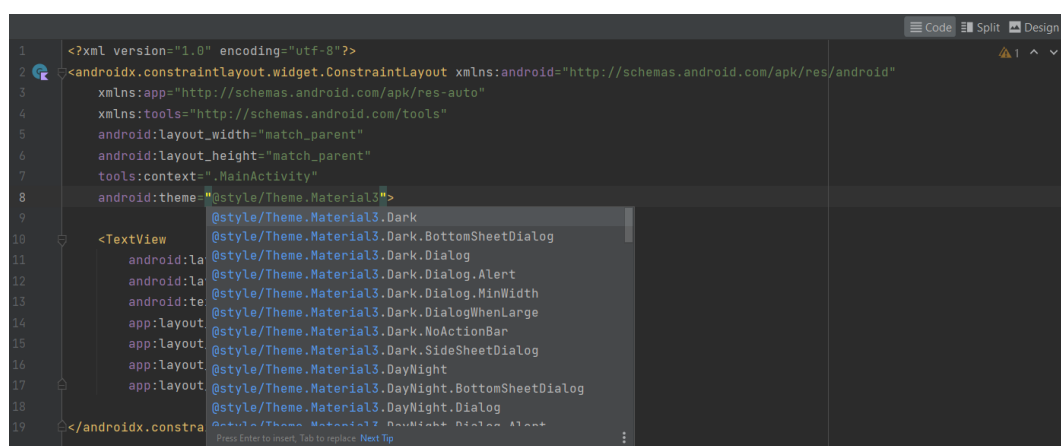
Po najechaniu kursorem na temat (*android:theme="@style/Theme.NazwaAplikacji*), można zapoznać się z kolorami oraz innymi ustawieniami, które w ramach tematu zostały tam uwzględnione. Domyślnie zestaw kolorów jest jak poniżej:

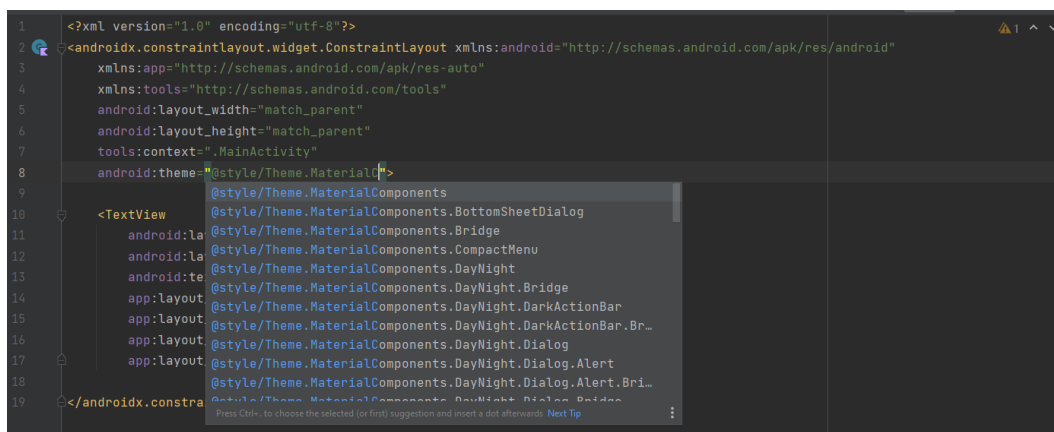


Po zmianie kolorów bazowych oraz pomocniczych, będzie to widoczne również w Maniście.

Styl

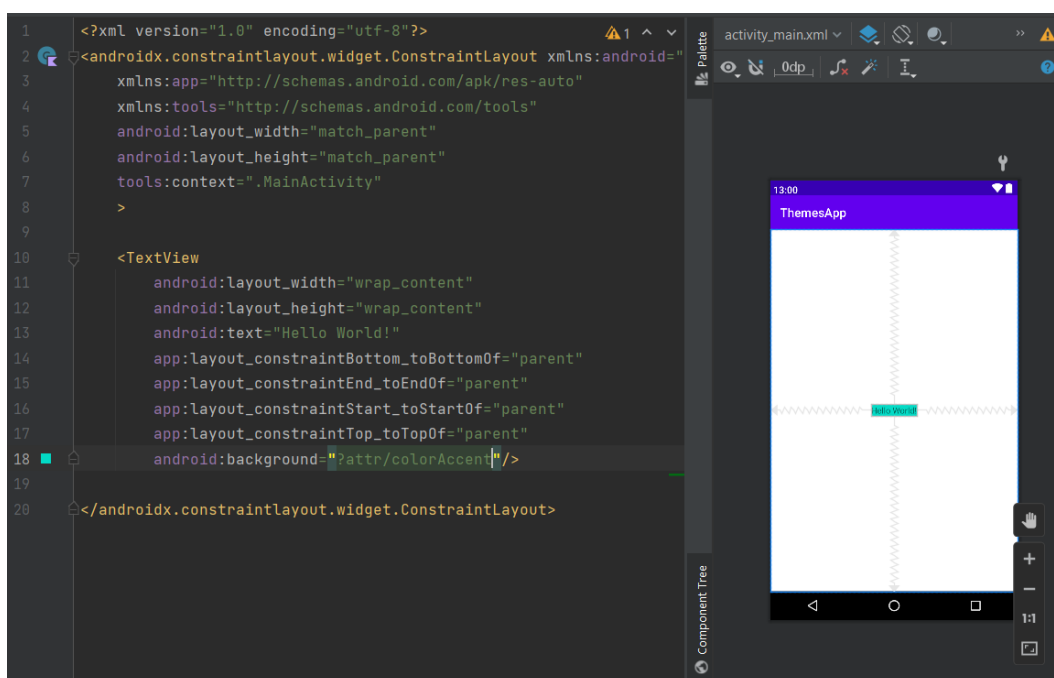
Style mogą dotyczyć zarówno całości layoutu i wszystkich komponentów, jak i ich wybranych części. Można zarówno utworzyć swój własny plik .xml w folderze themes (*res* → *values* → *themes* → *nazwa_pliku.xml*), jak i skorzystać z gotowych motywów (z bibliotek *Material3* albo *MaterialComponents*), jak poniżej.





Więcej na temat korzystania z biblioteki MaterialComponents znajduje się w dalszej części niniejszego przewodnika.

Można również zdefiniować bardziej elastyczne motywy i style, odwołując się do atrybutów motywu zamiast bezpośrednio do określonych zasobów. Aby odwołać się do atrybutu motywu należy zastosować składnię `?attr/<nazwa_atrybutu_motywu>`.



Inne atrybuty komponentów można dostosowywać i zmieniać dowolnie, mając świadomość, że właściwości tematów/motywów są nadpisywane przez style, które to z kolei są nadpisywane przez atrybuty poszczególnych komponentów.

W przypadku stworzenia własnego stylu, **wysoce zalecane jest stosowanie się do przyjętej konwencji**, tj. stworzenie pliku `styles.xml`. Najłatwiej to zrobić klikając PPM na katalog `values` → `New` → `Values Resource File`. Następnie można zdefiniować styl od podstaw (bazując przykładowo na pliku `themes.xml`) lub rozszerzyć istniejący styl z biblioteki MDC (Material Design Components). Określona konwencją jest również nazwa danego stylu – przykładowo, zastępując `MaterialComponents` nazwą swojej aplikacji. Dzięki temu eliminowane jest ryzyko

powstania konfliktów związanych z przestrzenią nazw (gdy MaterialComponents wprowadzi kolejne style).

Uwaga: Jeśli stylizujesz swoją aplikację, a rezultaty są niewidoczne, to najprawdopodobniej inne style nadpisują Twoje zmiany.

2. Material Components

Material Components to interaktywne bloki, które służą do tworzenia interfejsu użytkownika. Posiadają one wbudowany sposób komunikowania fokusu, informowania o wyborze. W czasie zajęć można korzystać z dodatkowej biblioteki Material Components for Android.

```
implementation 'com.google.android.material:material:<version>'
```

Tryb dzienny/nocny

Motyw DayNight umożliwia dodanie ciemnego motywu, gdy w urządzeniu włączony jest tryb nocny. W celu zdefiniowania innego motywu trybu nocnego, powinno się utworzyć nowy plik themes.xml w folderze values-night i określić pożądane atrybuty.

```
<style name="AppTheme" parent="Theme.MaterialComponents.DayNight">
  <item name="colorPrimary">@color/orange_500</item>
  ...
```

```
<style name="AppTheme" parent="Theme.MaterialComponents.DayNight">
  <item name="colorPrimary">@color/orange_200</item>
  ...
```

Dodatkowo, można zdefiniować motywy Light i Dark aplikacji poprzez dziedziczenie z Theme.MaterialComponents.Light w katalogu values i Theme.MaterialComponents w katalogu values-night. Np:

res/values/themes.xml

```
<style name="Theme.MyApp" parent="Theme.MaterialComponents.Light">
  <!-- ... -->
</style>
```

res/values-night/themes.xml

```
<style name="Theme.MyApp" parent="Theme.MaterialComponents">
  <!-- ... -->
</style>
```

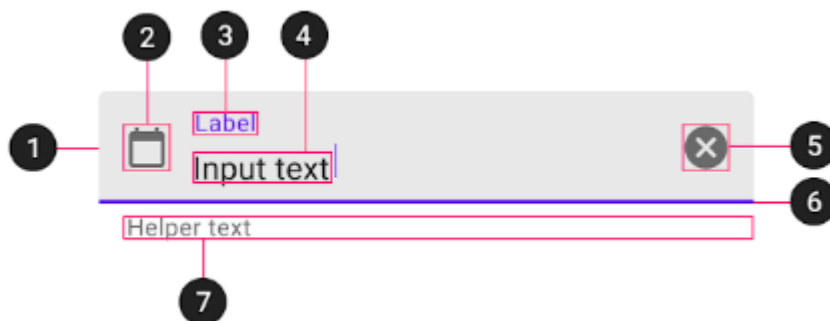
Motyw Theme.MaterialComponents jest statycznym motywem ciemnym, a Theme.MaterialComponents.DayNight jest bardziej dynamiczny, który umożliwia przełączanie między jasnym a ciemnym trybem aplikacji. Podczas korzystania z aplikacji DayNight można definiować jeden motyw aplikacji, który odwołuje się do zasobów kolorów, które można nadpisywać w katalogu values-night.

Pola tekstowe

Dotychczas podczas zajęć używano **EditText**, jednakże zalecane jest korzystanie z pól tekstowych Material, w których można wprowadzać, edytować oraz modyfikować tekst. Pole tekstowe Material składa się z *TextInputLayout* i *TextInputEditText*. Używanie *EditText* jako dziecka może działać, jednakże *TextInputEditText* zapewnia wsparcia dostępności dla pola tekstowego i pozwala *TextInputLayout* na kontrolę nad wizualnymi aspektami tekstu wejściowego.

Uwaga: Jeśli używany jest *EditText* należy ustawić `android:background` na `@null`, aby *TextInputLayout* mógł ustawić tło na nim.

Ładnie przygotowane pole tekstowe składa się z elementów przedstawionych na poniższym rysunku.



1. Container → kontener
2. Leading icon → wiodąca ikona
3. Label → etykieta
4. Input text → wprowadzany tekst
5. Trailing icon → ikona końcowa
6. Activation indicator → wskaźnik aktywacji
7. Helper/error text → tekst błędu / pomocy

Każdy z fragmentów 1-7 charakteryzuje się poszczególnymi atrybutami, które powinny być ustawione na *TextInputLayout* z wyjątkiem atrybutów wprowadzanego tekstu, które należy zdefiniować w *TextInputEditText*:

a. Container

Element	Atrybut	Wartość domyślana
Color (kolor)	app:boxBackgroundColor	?attr/colorOnSurface
Shape (kształt)	app:shapeAppearance	?attr/shapeAppearanceSmall Component
Text field enabled (włączone pole tekstowe)	android:enabled	true

b. Leading icon attributes

Element	Atrybut	Wartość domyślana
Icon (ikona)	app:startIconDrawable	null
Content description (opis zawartości)	app:startIconContentDescription	null
Color (kolor)	app:startIconTint	colorOnSurface

c. Label attributes

Element	Atrybut	Wartość domyślana
Text (tekst)	android:hint	null
Color (kolor)	android:textColorHint	?attr/colorOnSurface
Collapsed (floating) color (pływający kolor)	app:hintTextColor	?attr/colorPrimary
Typography (typografia)	app:hintTextAppearance	?attr/textAppearanceCaption
Animation (animacja)	app:hintAnimationEnabled	true

d. Input text attributes

Element	Atrybut	Wartość domyślana
Input text (tekst wprowadzany)	android:text	null
Typography (typografia)	android:textAppearance	?attr/textAppearanceSubtitle1

Input text color (kolor tekstu wprowadzanego)	android:textColor	?android:textColorPrimary

e. Trailing icon attributes

Element	Atrybut	Wartość domyślana
Mode (tryb)	app:endIconMode	END_ICON_NONE
Color (kolor)	app:endIconTint	ColorOnSurface colorPrimary
Custom icon (ikona)	app:endIconDrawable	null
Custom icon content description (opis ikony)	app:endIconContentDescription	null
Custom icon checkable (ikona sprawdzenia)	app:endIconCheckable	true

f. Activation indicator attributes

Element	Atrybut	Wartość domyślana
Color (kolor)	app:boxStrokeColor	?attr/colorOnSurface
Error color (kolor błędu)	app:boxStrokeErrorColor	?attr/colorError
Width (szerokość)	app:boxStrokeWidth	1 dp
Focused width	app:boxStrokeWidthFocused	2 dp

g. Helper/error text

Element	Atrybut	Wartość domyślana
Helper text enabled (tekst pomocniczy włączony)	app:helperTextEnabled	false
Helper text (tekst pomocniczy)	app:helperText	null
Helper text color (kolor tekstu)	app:helperTextColor	?attr/colorOnSurface

Helper text typography (tekst pomocniczy typografia)	app:helperTextAppearance	?attr/textAppearanceCaption
Error text enabled (tekst błędu włączony)	app:errorEnabled	false
Error text (tekst błędu)	N/A	null
Error text color (kolor tekstu)	app:errorTextColor	?attr/colorError
Error text typography (topografia tekstu błędu)	app:errorTextAppearance	?attr/textAppearanceCaption

3. Rotacja ekranu

Podczas obracania urządzenia może się zdarzyć tak, że część zawartości aplikacji nie będzie widoczna. Wówczas należy skorzystać z *ScrollView*, czyli specjalnego rodzaju layoutu, który umożliwia przewijanie zawartości. Należy zatem wykonać zamianę layoutu w pliku xml na *ScrollView*.

Warto zwrócić uwagę jeszcze na komponent *HorizontalScrollView*, ponieważ umożliwia on przewijanie lewo-prawo.

4. Ukrycie klawiatury po wciśnięciu Enter

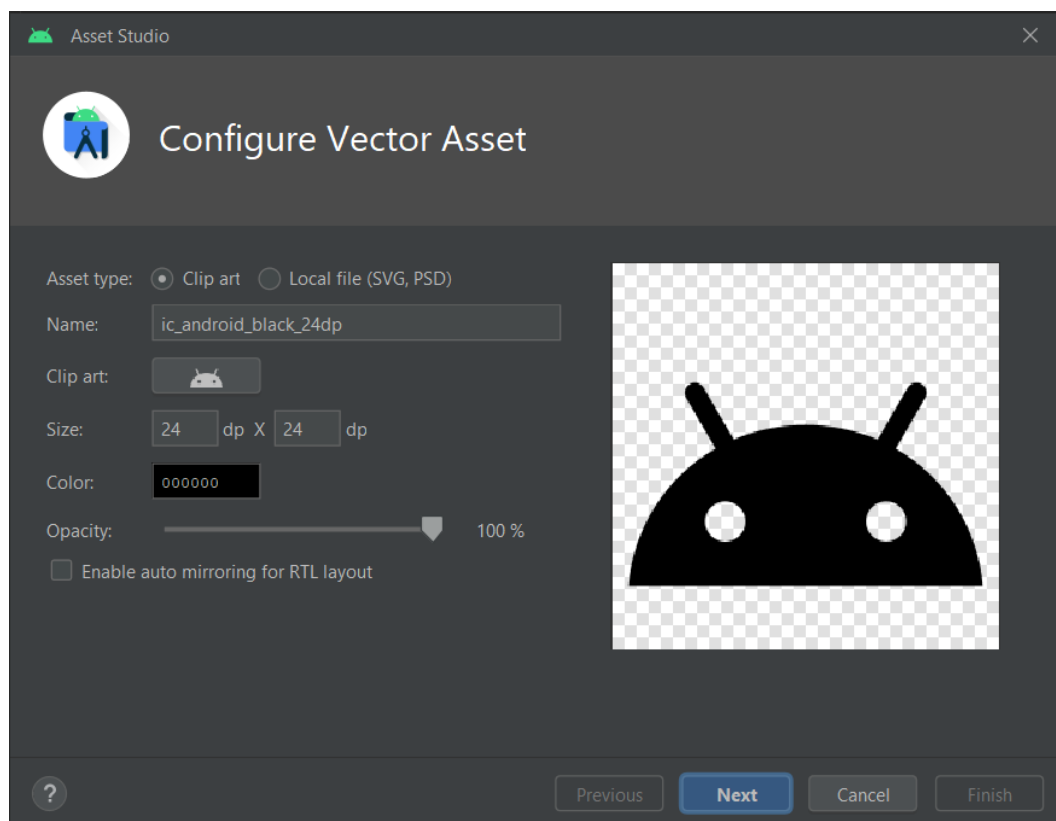
Należy zdefiniować nasłuchiwanie, kiedy zostanie wciśnięty przycisk Enter. Przydatna może być funkcja *handleKeyEvent()*, która ukrywa klawiaturę ekranową, jeśli *keyCode* jest równy *KeyEvent.KEYCODE_ENTER*.

Metoda *InputMethodManager* zwraca wartość *true*, jeśli kluczowe zdarzenie zostało obsłużone, w przeciwnym razie zwraca wartość *false*.

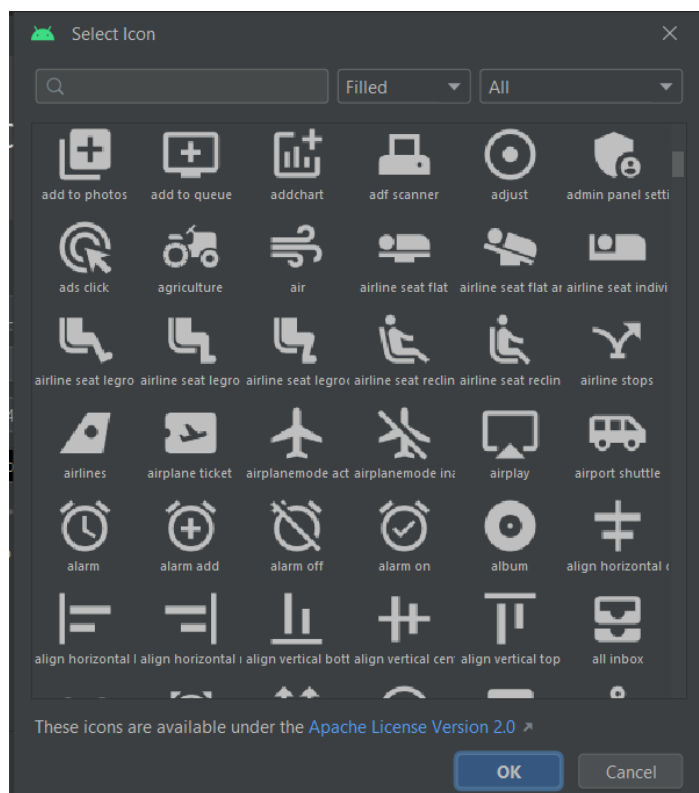
```
private fun handleKeyEvent(view: View, keyCode: Int): Boolean {
    if (keyCode == KeyEvent.KEYCODE_ENTER) {
        // Hide the keyboard
        val inputMethodManager =
            getSystemService(Context.INPUT_METHOD_SERVICE) as
            InputMethodManager
        inputMethodManager.hideSoftInputFromWindow(view.windowToken, 0)
        return true
    }
    return false
}
```

5. Dodawanie ikon

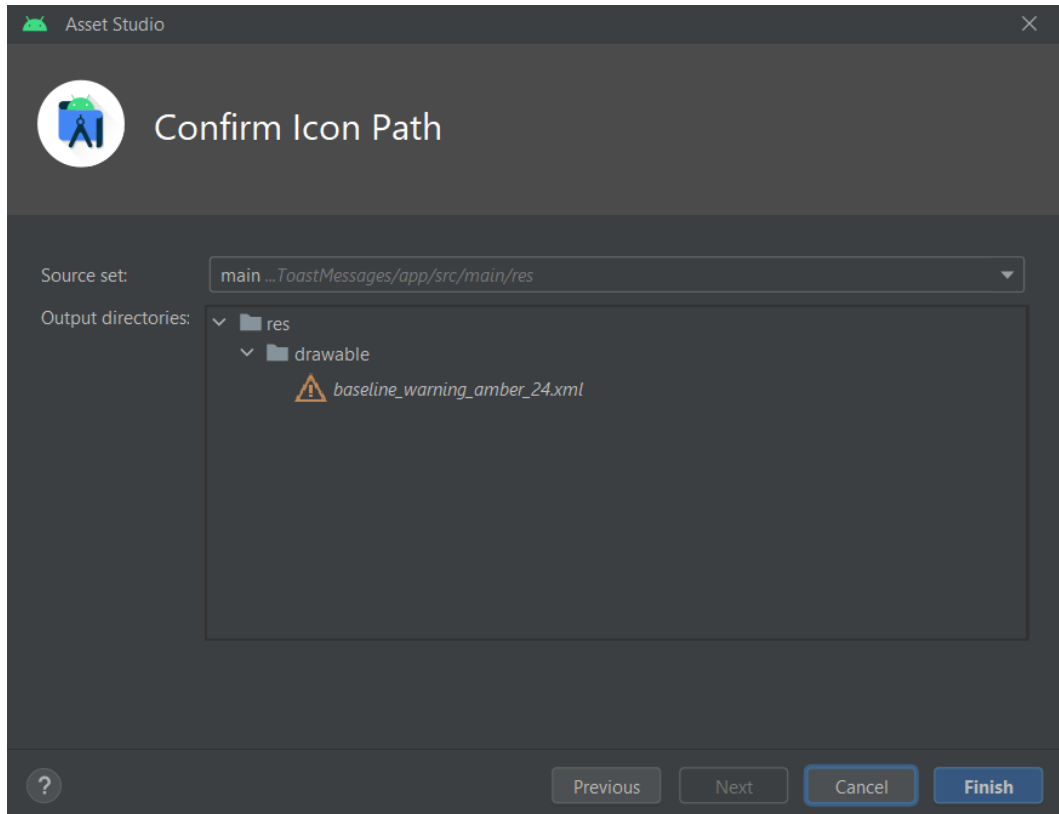
➔ W drzewie projektu kliknąć PPM na folder *drawable* ➔ *New* ➔ *Vector Asset*



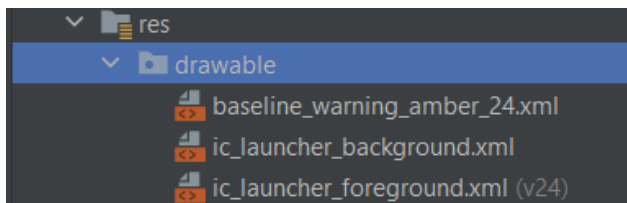
➔ Po kliknięciu na *Clip art* wybrać odpowiednią ikonkę pasującą do treści komponentu:



- ➔ Można sobie wybrać również kolor ikonki. Po ustawieniu i kliknięciu OK pojawia się takie okno:



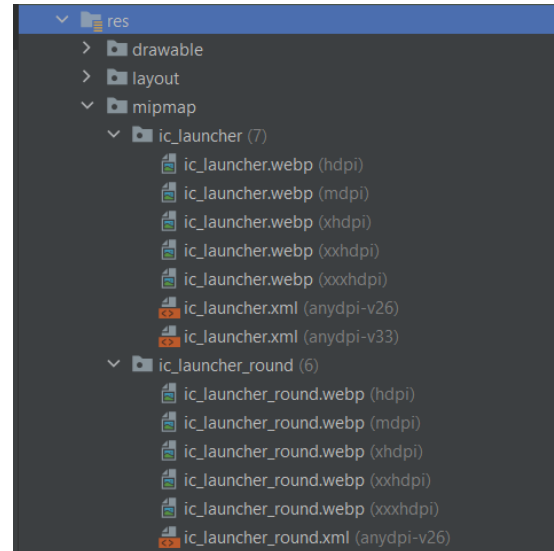
- ➔ Po kliknięciu Finish, w folderze drawable jest do dyspozycji w ramach projektu opracowana ikonka.



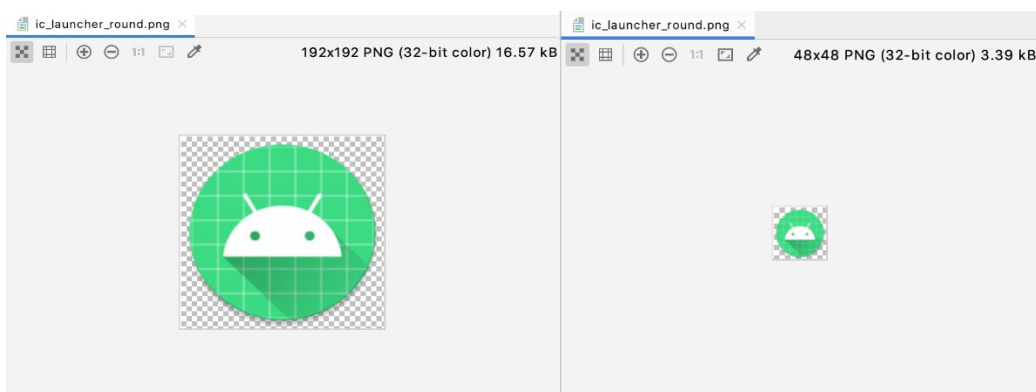
6. Ikona aplikacji

Wstęp

Ikona aplikacji stanowi element graficzny, który jako pierwszy ma kontakt z użytkownikiem. Z tego względu warto zastanowić się, w jaki sposób może odzwierciedlać aplikację (lub reprezentować daną markę). Z technicznego punktu widzenia **ikona aplikacji powinna być ostra i wyraźna, niezależnie od modelu urządzenia lub gęstości ekranu**. Kwalifikatory gęstości określają liczbę punktów (pikseli, kropek) na cal (*dots per inch, dpi*). Podczas tworzenia aplikacji należy zadbać, aby aplikacja była wyposażona w reprezentację graficzną ikon o zróżnicowanej gęstości. To podejście znajduje swoje odzwierciedlenie w drzewie projektu po utworzeniu *Empty Activity* za pomocą kreatora Android Studio (folder *res* → *mipmap* → *ic_launcher* lub *ic_launcher_round*).

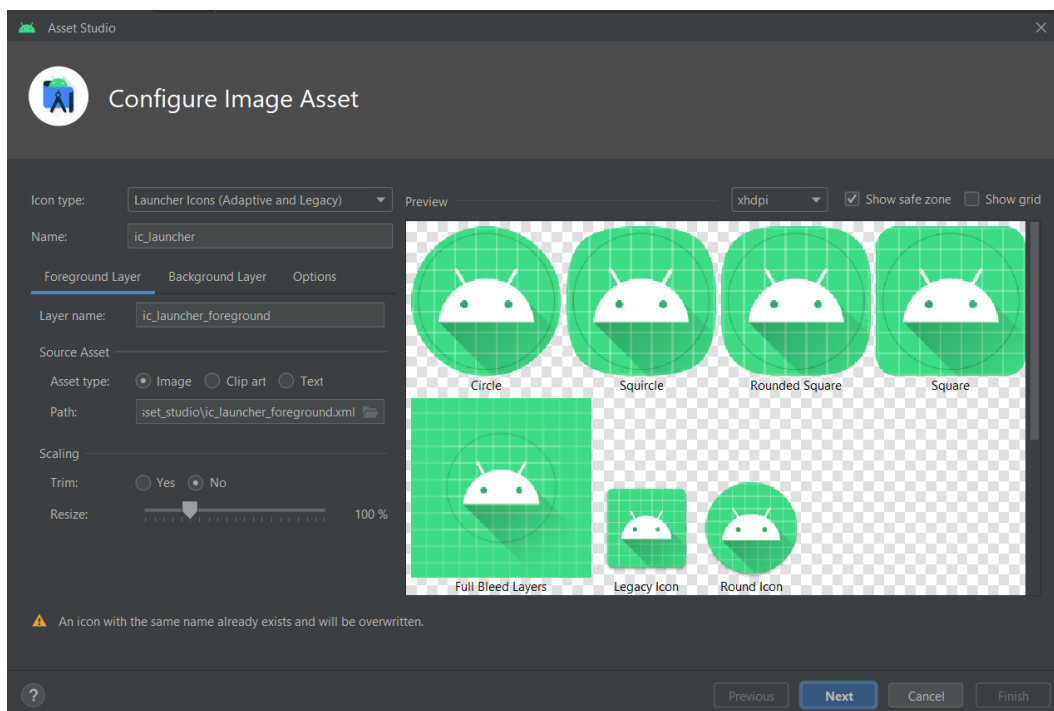


Po podwójnym kliknięciu na poszczególne pliki graficzne można zauważyć znaczącą różnicę w wymiarach obrazów, tak jak poniżejⁱⁱⁱ.

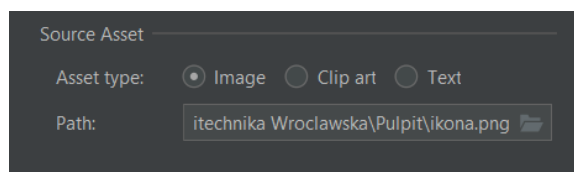


Tworzenie ikony

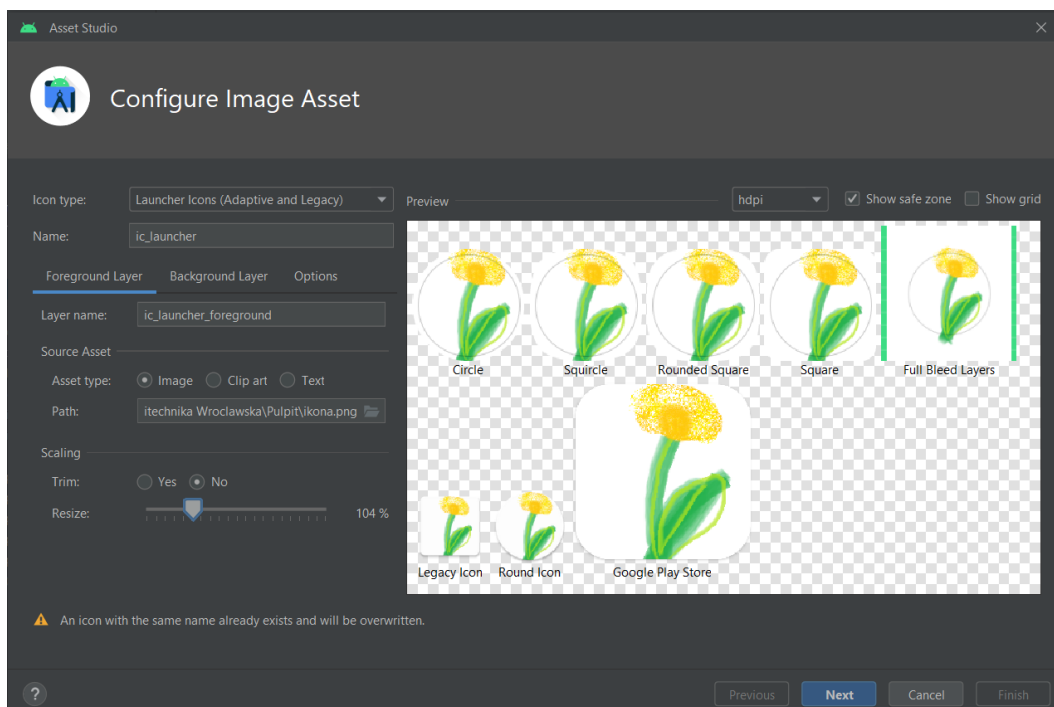
W celu **utworzenia nowej ikony** należy wykorzystać narzędzie *Configure Image Asset* (PPM na drzewie projektu → *New* → *Image asset*)



Celem **ustawienia własnej ikony**, należy wybrać ścieżkę pliku, który ma tę ikonę stanowić:

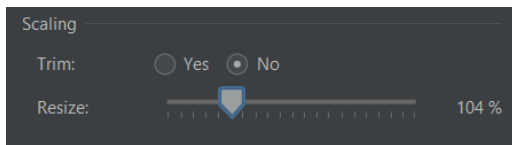


Po podmienieniu pliku, w oknie nastąpi zmiana grafiki:

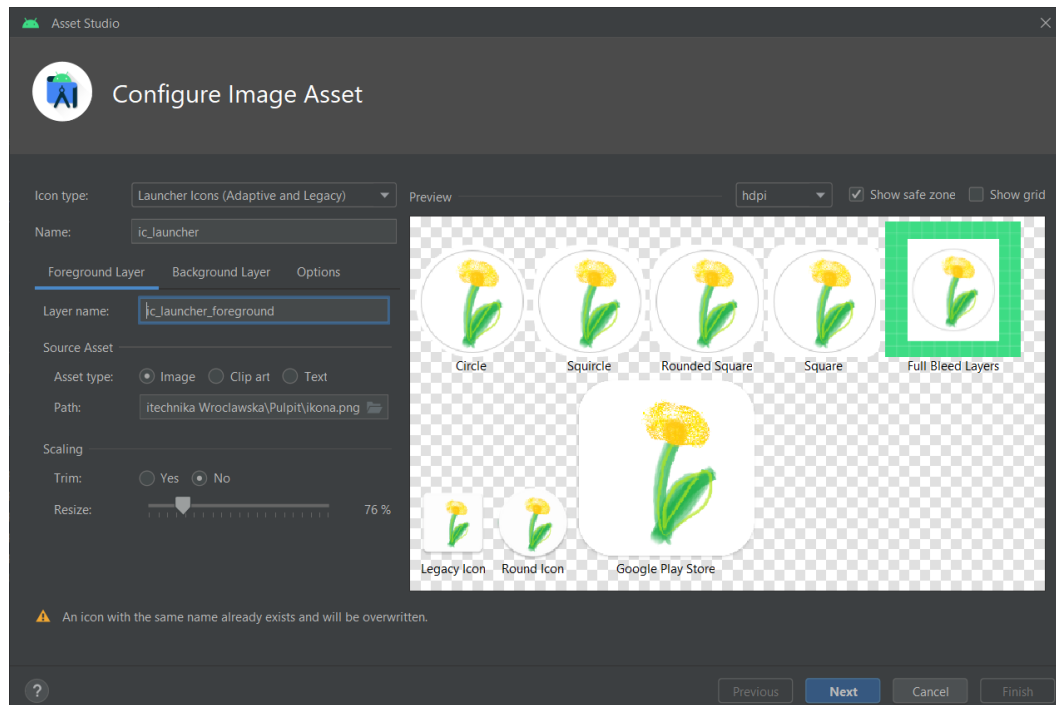


Po ustawieniu, należy się upewnić, **czy obraz mieści się w tak zwanej „strefie bezpiecznej”** (*safe zone*, zwizualizowanej za pomocą szarego okręgu na ikonkach). Celem sprawdzenia należy

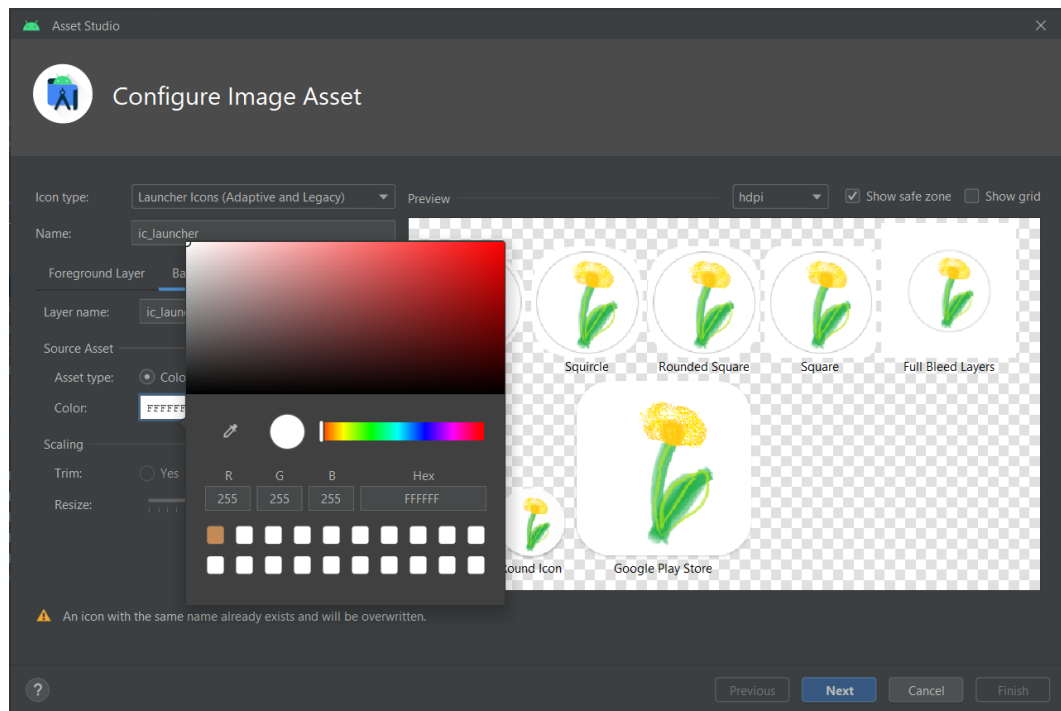
zaznaczyć odpowiedni check box w prawym górnym rogu (☒ Show safe zone). Aby poprawić dopasowanie, konieczne jest wykorzystanie suwaka przy części *Scaling*.



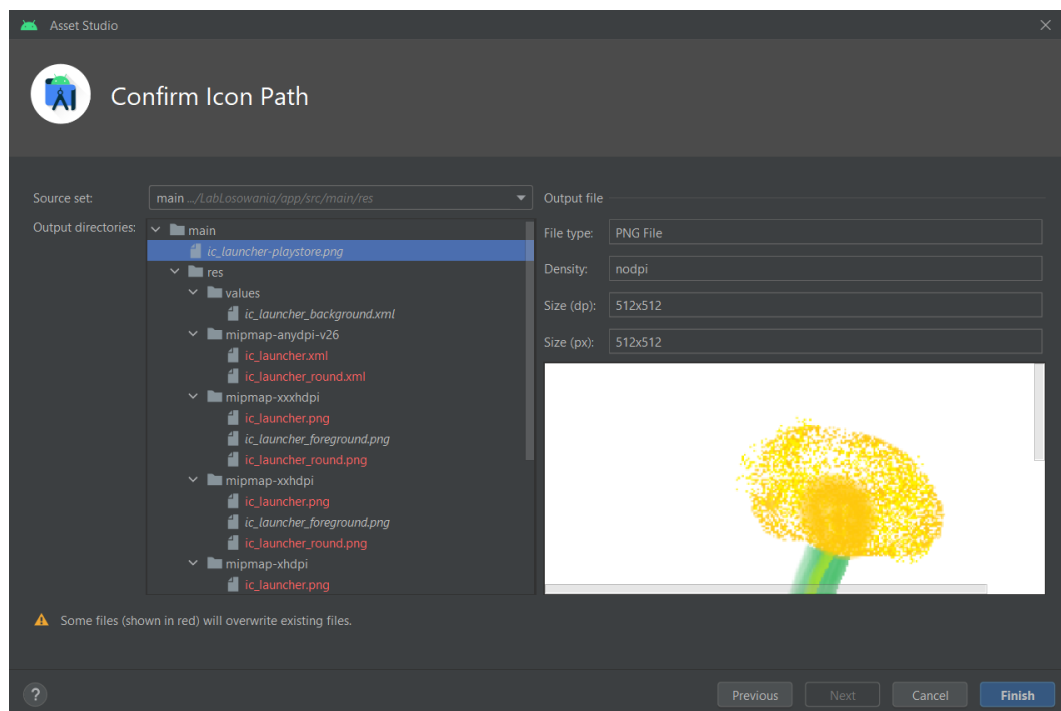
Po odpowiednim przeskalowaniu obrazu, gotowy projekt części przedniej (**Foreground Layer**) ikony aplikacji powinien wyglądać jak poniżej:



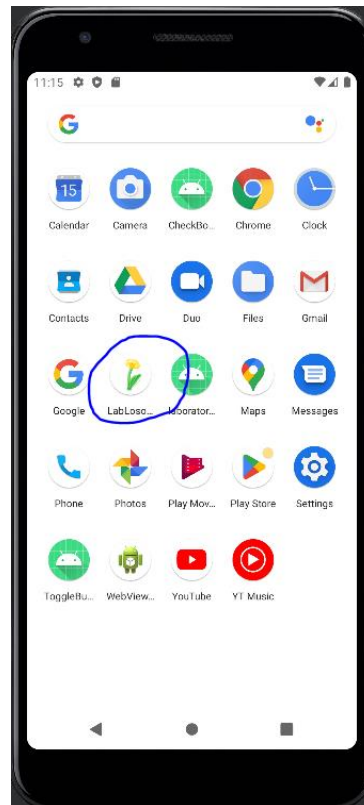
Następnie należy przejść do części *Background Layer* i tam zmienić obraz źródłowy lub kolor na odpowiadający projektowi. W niniejszym przypadku zmieniono kolor tła na biały (po kliknięciu na obszar z kolorem oraz kodem koloru w części *Source Asset* otwiera się okno, w którym można dobrać odpowiedni kolor tła w spodniej warstwie), otrzymując efekt jak zaprezentowano:



Po kliknięciu przycisku *Next* wyświetli się monit o **nadpisaniu plików** (zaznaczonych na czerwono).



Następnie klikając *Finish* oraz reinstalacji aplikacji na emulatorze/urządzeniu, **ikona zmieni się na nową**.



Źródła i przydatne linki:

<https://developer.android.com/develop/ui/views/theming/themes>

<https://medium.com/androiddevelopers/android-styling-themes-vs-styles-ebe05f917578>

<https://developer.android.com/develop/ui/views/theming/themes>

<https://developer.android.com/develop/ui/views/text-and-emoji/downloadable-fonts>

<https://www.figma.com/community/plugin/1034969338659738588/Material-Theme-Builder>

<https://forum.android.com.pl/topic/354398-dodawanie-g%C5%82%C3%B3wnej-ikony-do-aplikacji>

ⁱ Odwołując się do analogii z OOP, motyw można utożsamiać z interfejsem, a styl z klasą dziedziczącą po tym interfejsie. Więcej na wykładzie kursu *Android*

ⁱⁱ Opisane w dalszej części instrukcji

ⁱⁱⁱ Więcej informacji na temat ikon, ich rozdzielczości oraz zasadności stosowania różnych *dpi jest podane na wykładzie kursu *Android*